

Deep Learning

Generative Adversarial Network (GAN)

Generative Adversarial Networks (GAN)

① Work by Ian Goodfellow et al. ([NeurIPS 2014](#))

Goal

- ① Sampler that draws high quality samples from p_m

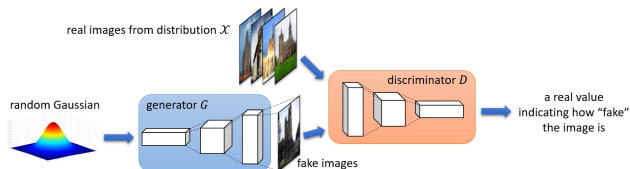
Goal

- ① Sampler that draws high quality samples from p_m
- ② Without computing p_x and p_m ensures closeness

Goal

- ① Sampler that draws high quality samples from p_m
- ② Without computing p_x and p_m ensures closeness
- ③ Draws samples that are similar to the training data (but not exactly them)

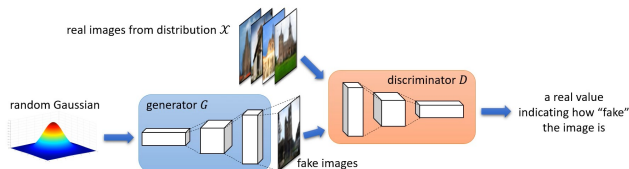
Method



Credit: Microsoft research blog

- 1 Introduce a latent variable (z) with a simple prior (p_z)

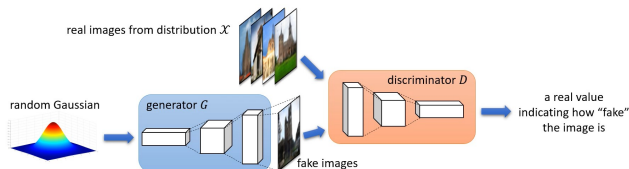
Method



Credit: Microsoft research blog

- ① Introduce a latent variable (z) with a simple prior (p_z)
- ② Draw $z \sim p_z$, i/p to the generator (G) $\rightarrow \hat{x} \sim p_G$

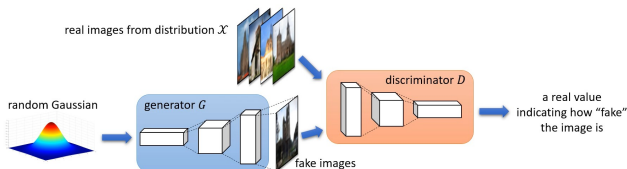
Method



Credit: Microsoft research blog

- ① Introduce a latent variable (z) with a simple prior (p_z)
- ② Draw $z \sim p_z$, i/p to the generator (G) $\rightarrow \hat{x} \sim p_G$
- ③ Machinery to ensure $p_G \approx p_{\text{data}}$

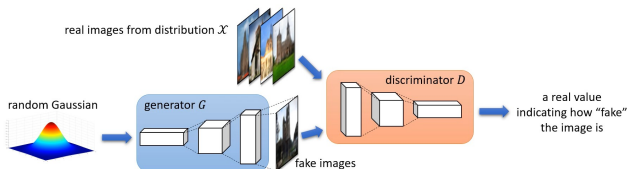
$$p_G \approx p_{\text{data}}$$



Credit: Microsoft research blog

- ① Employ a classifier to differentiate between **real** samples $x \sim p_{\text{data}}$ (label 1) and **generated**(fake) ones $\hat{x} \sim p_G$ (label 0)

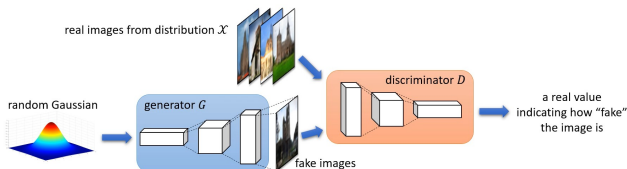
$$p_G \approx p_{\text{data}}$$



Credit: Microsoft research blog

- ① Employ a classifier to differentiate between **real** samples $x \sim p_{\text{data}}$ (label 1) and **generated**(fake) ones $\hat{x} \sim p_G$ (label 0)
- ② Referred to as the **Discriminator (D)**

$$p_G \approx p_{\text{data}}$$



Credit: Microsoft research blog

- ① Employ a classifier to differentiate between **real** samples $x \sim p_{\text{data}}$ (label 1) and **generated**(fake) ones $\hat{x} \sim p_G$ (label 0)
- ② Referred to as the **Discriminator (D)**
- ③ Train the G such that D misclassifies generated samples $\hat{x}$ into class 1 (can't differentiate b/w $x \sim p_{\text{data}}$ and $\hat{x} \sim p_G$)

Training Objective

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \right)$$

- ① minmax optimization (or, zero-sum game)

Training Objective

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \right)$$

- ① minmax optimization (or, zero-sum game)
- ② With a sigmoid o/p neuron, $D(\cdot) \rightarrow$ probability that the i/p is real

Training Objective

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \right)$$

- ① minmax optimization (or, zero-sum game)
- ② With a sigmoid o/p neuron, $D(\cdot) \rightarrow$ probability that the i/p is real
- ③ Expectation in practice is average over a batch of samples

Training Strategy

- ① Natural idea is to go for training D first and then to train G

Training Strategy

- ① Natural idea is to go for training D first and then to train G
- ② Issue here would be poor gradients for training G .

Training Strategy

- ① Natural idea is to go for training D first and then to train G
- ② Issue here would be poor gradients for training G .
- ③ $\min_G \left(\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \right)$

Training Strategy

- ① Natural idea is to go for training D first and then to train G
- ② Issue here would be poor gradients for training G .
- ③ $\min_G \left(\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \right)$
- ④ For an x generated by G , $\frac{\partial \log(1 - \sigma(x))}{\partial x} = \frac{\sigma(x) \cdot (\sigma(x) - 1)}{(1 - \sigma(x))} = -\sigma(x)$

Training Strategy

- ① Natural idea is to go for training D first and then to train G
- ② Issue here would be poor gradients for training G .
- ③ $\min_G \left(\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \right)$
- ④ For an x generated by G , $\frac{\partial \log(1 - \sigma(x))}{\partial x} = \frac{\sigma(x) \cdot (\sigma(x) - 1)}{(1 - \sigma(x))} = -\sigma(x)$
- ⑤ Which would be ≈ 0 for a confident $D \rightarrow$ (no gradients to train G !)

Training Strategy

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log \left(1 - D(G(\mathbf{z}^{(i)})) \right) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(\mathbf{z}^{(i)})) \right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Optimality

① $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$ (Optimal Discriminator for any G)

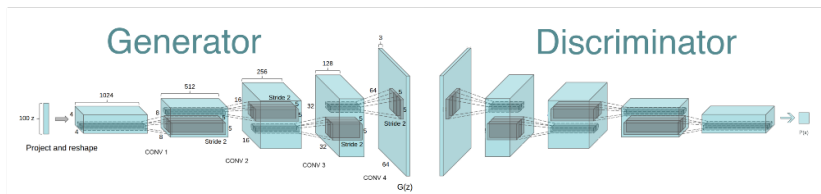
Optimality

- ① $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$ (Optimal Discriminator for any G)
- ② $p_{\text{data}} = p_G$ (Optimal Generator for any D)

Optimality

- ① $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$ (Optimal Discriminator for any G)
- ② $p_{\text{data}} = p_G$ (Optimal Generator for any D)
- ③ $D_G^*(x) = \frac{1}{2}$

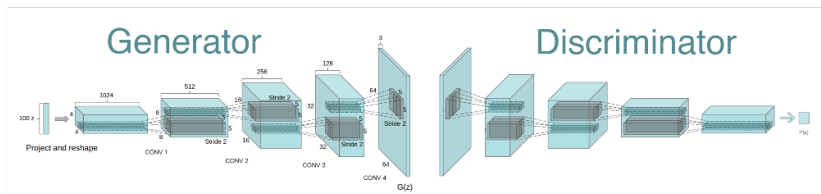
Deep Convolutional GAN (DC-GAN)



Radford et al. ICLR 2016

- ① Combined the developments of CNNs with the generative modeling

Deep Convolutional GAN (DC-GAN)

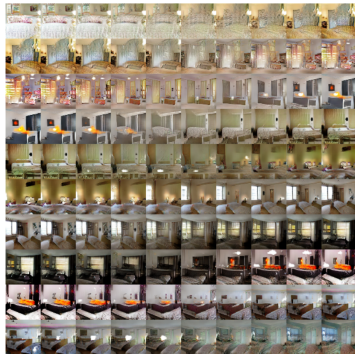


Radford et al. ICLR 2016

- ① Combined the developments of CNNs with the generative modeling
- ② Demonstrated some of the best practices for stable training of deep GAN architectures

Moving in the latent space

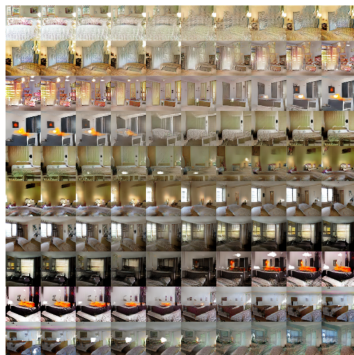
- ① Interpolate between two points in the latent space and visualize



Radford et al. ICLR 2016

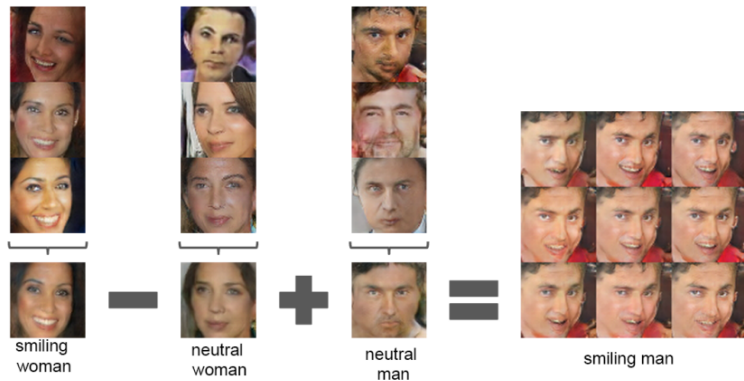
Moving in the latent space

- ① Interpolate between two points in the latent space and visualize
- ② Smooth transition in the generated image is a sign of good model



Radford et al. ICLR 2016

Vector arithmetic



Radford et al. ICLR 2016

Pose Transformation



Radford et al. ICLR 2016

Representation learning

Table 1: CIFAR-10 classification results using our pre-trained model. Our DCGAN is not pre-trained on CIFAR-10, but on Imagenet-1k, and the features are used to classify CIFAR-10 images.

Model	Accuracy	Accuracy (400 per class)	max # of features units
1 Layer K-means	80.6%	63.7% ($\pm 0.7\%$)	4800
3 Layer K-means Learned RF	82.0%	70.7% ($\pm 0.7\%$)	3200
View Invariant K-means	81.9%	72.6% ($\pm 0.7\%$)	6400
Exemplar CNN	84.3%	77.4% ($\pm 0.2\%$)	1024
DCGAN (ours) + L2-SVM	82.8%	73.8% ($\pm 0.4\%$)	512

Radford et al. ICLR 2016

Evaluating GANs

- ① Open research problem

Evaluating GANs

- ① Open research problem
- ② Humans judgement!

Evaluating GANs

- ① Open research problem
- ② Humans judgement!
- ③ In case of images
 - **Recognizable objects:** accurate and high-confidence predictions by a classifier
 - **Semantic diversity:** samples should be drawn evenly from all categories of train data

Inception Score (IS)

- ① Consider the pretrained Inception classifier $\rightarrow p(y/x)$

Inception Score (IS)

- ① Consider the pretrained Inception classifier $\rightarrow p(y/x)$
- ② label distribution of the generated samples $\rightarrow p(y)$

Inception Score (IS)

- ① Consider the pretrained Inception classifier $\rightarrow p(y/x)$
- ② label distribution of the generated samples $\rightarrow p(y)$
- ③ Desired: low entropy for $p(y/x)$ (distinctly recognizable) and high entropy for $p(y)$ (semantic diversity)

Inception Score (IS)

- ① Consider the pretrained Inception classifier $\rightarrow p(y/x)$
- ② label distribution of the generated samples $\rightarrow p(y)$
- ③ Desired: low entropy for $p(y/x)$ (distinctly recognizable) and high entropy for $p(y)$ (semantic diversity)
- ④ Inception score (IS) = $\exp \left(H(y) - H(y/x) \right)$

Inception Score (IS)

- ① Consider the pretrained Inception classifier $\rightarrow p(y/x)$
- ② label distribution of the generated samples $\rightarrow p(y)$
- ③ Desired: low entropy for $p(y/x)$ (distinctly recognizable) and high entropy for $p(y)$ (semantic diversity)
- ④ Inception score (IS) = $\exp \left(H(y) - H(y/x) \right)$
- ⑤ Higher is better